

MAS Design Workshop

Example Answer Guide

Agentic AI Coding Fitness | Week 9 | All 10 Scenarios + Reflection

This guide provides worked example answers for all 8 design questions across every scenario. Share with participants AFTER the workshop, or use as a teaching reference during facilitation.

Contents: 10 Scenarios × Q1–Q8 Example Answers | Workshop Reflection Answers | Key Insights

How to Use This Guide

During the workshop	Keep this guide closed. Let participants struggle with the design questions first — that struggle is where the learning happens.
When someone is stuck	You may reveal the Q1 example answer for their scenario to help them frame the problem correctly, then let them continue independently.
After the workshop	Distribute this PDF. Participants compare their designs to the examples and identify differences — differences are learning opportunities, not mistakes.
No perfect answer	These are strong example answers, not the only correct answers. A participant's design may differ and still be valid — encourage debate.
Key insight boxes	Look for the orange "Key insight" boxes — these highlight the single most important lesson from each question.

Q1 What specific problem are you solving?

Sample problem statement:

A 300-room hotel with 12 function halls wastes an estimated 40% of its energy budget because HVAC, lighting, elevators, and pool heating operate as completely isolated systems with no shared data or coordination. The goal is to reduce energy cost by 30% (≈THB 120,000/day savings) while maintaining a guest comfort score of $\geq 4.5/5$ and responding to occupancy changes within 60 seconds.

Key insight: A strong problem statement names WHO is affected, WHAT is failing, and WHAT "success" looks like in measurable terms. Note the THB number and the comfort KPI — without these, it is impossible to know if your MAS actually works.

Q2 Why do you need MAS and not a single agent?

Why MAS is justified — at least 2 of these 4 criteria apply:

- The problem has distinct parallel subsystems (HVAC zones, lighting floors, pool): a single agent cannot efficiently manage all in one context window.
- Specialization: OccupancyAgent needs sensor-fusion logic; HVACAgent needs thermodynamics — fundamentally different tools and knowledge.
- Continuous parallelism: 12 zones require real-time simultaneous monitoring; sequential processing would introduce unacceptable delay.
- Fault tolerance: if LightingAgent crashes, HVAC control must continue unaffected — a single agent would fail entirely.

Key insight: If you answered "yes" to at least 2 of these 4 criteria (parallel, specialized, fault-tolerant, too large), a MAS is justified. For this scenario all 4 apply clearly.

Q3 How many agents do you need?

5 agents — one per distinct responsibility boundary:

Agent Name	Primary Role
OccupancyAgent	Predicts room/zone occupancy 30 min ahead using sensor + booking data
HVACControlAgent	Adjusts temperature setpoints per zone based on occupancy forecast
LightingAgent	Controls light intensity and schedule per floor/zone
EnergyBudgetAgent	Enforces daily energy budget across all systems — the orchestrator
WeatherForecastAgent	Fetches 24-hr weather data to pre-cool/pre-heat rooms proactively

Key insight: 5 agents is appropriate here. Resist the urge to add agents per room (300 agents!). The boundary is by subsystem responsibility, not by physical unit.

Q4 What does each agent do?

For each agent — goal, tools, output, and failure behavior:

Agent	Goal	Key Tools	Output	If it Fails
OccupancyAgent	Predict occupancy 30 min ahead	read_pms_bookings(), read_motion_sensors(), read_key_card_events()	occupancy_forecast_ _json	Budget Agent falls back to last known occupancy
HVACControlAgent	Maintain target temp per zone at minimum energy	read_zone_sensors(), set_hvac_setpoint(), read_weather_forecast()	control_command	HVACs hold last setpoint (safe default)
LightingAgent	Set light levels per floor based on occupancy + daylight	read_lux_sensors(), set_dimmer_level()	dimmer_commands	Lights stay at current level
EnergyBudgetAgent	Ensure total energy stays within daily budget (orchestrator)	read_energy_meters(), calculate_budget_allocation(), send_directive()	budget_allocation per agent	Alert engineering team; other agents self-manage
WeatherForecastAgent	Provide 24-hr weather data for proactive control	call_openweather_api()	weather_forecast_json	HVACControlAgent uses historical seasonal averages

Key insight: Always answer "what happens if this agent FAILS?" — it forces you to build a resilient design from the start, not as an afterthought.

Q5 How will agents communicate?**Communication Language:**

Structured JSON (machine-to-machine; no LLM reasoning needed for sensor telemetry). OPC-UA protocol for direct BMS integration.

Ontology (shared vocabulary):

Component	Domain Objects / Terms
Objects (Nouns)	Zone, Room, HVAC, Sensor, EnergyBudget, WeatherForecast
Actions (Verbs)	set_temperature(), read_occupancy(), adjust_dimmer(), fetch_forecast()
Attributes	zone_id, setpoint_celsius, current_temp_c, occupancy_count, budget_remaining_kwh
Constraints	temp [20, 26]°C; budget_remaining_kwh ≥ 0; comfort_score ≥ 4.5

FIPA-ACL Protocol — key message exchanges:

- OccupancyAgent → EnergyBudgetAgent: INFORM(occupancy_forecast)
- EnergyBudgetAgent → HVACControlAgent: REQUEST(optimize_zone, zone=lobby, budget=50kWh)
- HVACControlAgent → EnergyBudgetAgent: INFORM-DONE(zone=lobby, setpoint=22°C, saving=12kWh)
- EnergyBudgetAgent → All Zone Agents: CFP(task=reduce_load, target=15%) — when over budget
- Zone Agents → EnergyBudgetAgent: PROPOSE(my_saving=X%) or REFUSE(reason=comfort_constraint)

Key insight: Use INFORM for sensor readings (no response needed). Use REQUEST for control commands (need acknowledgment). Use CFP only when you need to negotiate among multiple agents.

Q6 How do agents collaborate?**Collaboration mode and rationale:**

Cooperative by default — all agents share the energy-saving goal. When the budget is over-limit, EnergyBudgetAgent runs a Contract Net Protocol: each zone agent proposes how much it can save, and EnergyBudgetAgent accepts the most cost-effective proposals. This is a market mechanism within an otherwise cooperative system.

Key insight: Most real MAS are cooperative by default but use competitive/market mechanisms for resource allocation sub-problems. You do not have to choose one mode globally.

Q7 What is the agents' environment?**Observability analysis:**

Partially observable — each zone agent sees only its own sensors. EnergyBudgetAgent has broader but still incomplete view (aggregated meters, not per-room occupancy). Dynamic — occupancy changes minute-by-minute; weather changes hourly. State estimation needed: OccupancyAgent forecasts 30 min ahead using time-series model trained on historical booking + motion data. Memory: OccupancyAgent needs 30-day episodic memory of occupancy patterns by day-of-week and time-of-day.

Key insight: Partial observability is the normal case in IoT/physical systems. Your design must account for the gap between what an agent can OBSERVE and what it needs to KNOW.

Q8 How does MAS fit your frontend and backend?**Frontend / Backend / Deployment design:**

API surface: REST GET /api/energy/status (dashboard polling) + WebSocket stream for real-time meters.

Trigger: Agents run autonomously on a 60-second sensor cycle. Dashboard also provides a Manual Override button (POST /api/energy/manual_override).

Reporting: EnergyBudgetAgent pushes daily energy report to property management dashboard via REST callback.

Deployment: Docker containers on an edge server at the hotel (low latency to BMS). EnergyBudgetAgent is the central orchestrator.

ADK Pattern: LoopAgent(60s sensor cycle) wrapping ParallelAgent([OccupancyAgent, WeatherAgent]) → EnergyBudgetAgent → ParallelAgent([HVACControlAgent, LightingAgent]).

Monitoring: Each agent logs heartbeat + action_count to TimescaleDB. Alert if no heartbeat in 5 minutes.

Key insight:

For IoT MAS, deploy on EDGE (low latency) not cloud. LoopAgent is the right ADK pattern for continuous sensor-driven systems.

Q1 What specific problem are you solving?**Sample problem statement:**

An online marketplace processing 50,000 orders/day across 20 warehouses cannot maintain its 98% on-time delivery SLA because inventory is fragmented, routing is manual, and demand spikes of 10x occur with no warning. The goal is a MAS that validates, allocates, routes, and dispatches each order within 5 minutes while keeping fulfillment cost below THB 85/order and on-time delivery above 98%.

Key insight: Good problem statements quantify both the failure (current state) and the success criteria (target KPIs). "Manual routing" alone is too vague — "routing takes 20 minutes and causes 2% late deliveries" is specific.

Q2 Why do you need MAS and not a single agent?**Why MAS is justified — at least 2 of these 4 criteria apply:**

- Sequential pipeline: validation → inventory → routing → delivery → notification must run in order but each step is a different domain.
- Scale: 50,000 orders/day ≈ 35 orders/minute requires parallel processing; a single agent cannot keep up.
- Specialization: inventory optimization, logistics routing, customer communication are completely different skill domains.
- Fault tolerance: if DeliveryAgent fails for one carrier, other carriers and warehouse steps continue.

Key insight: A sequential pipeline MAS is still a MAS — the key justification is specialization + fault isolation at each step, not just parallelism.

Q3 How many agents do you need?**5 agents — one per distinct responsibility boundary:**

Agent Name	Primary Role
OrderIntakeAgent	Validates and classifies incoming orders (payment, SLA, type)
InventoryAgent	Checks stock levels across all warehouses and reserves items
WarehouseRouterAgent	Selects optimal warehouse per order (proximity + stock + SLA)
DeliveryAgent	Assigns delivery partner, calculates route and ETA
CustomerNotifyAgent	Sends order confirmation, tracking links, and delivery updates

Key insight: The pipeline maps directly to: Intake → Stock → Where → How → Who. Each stage has a clear input and output — that is how to identify the right agent boundaries.

Q4 What does each agent do?**For each agent — goal, tools, output, and failure behavior:**

Agent	Goal	Key Tools	Output	If it Fails
OrderIntakeAgent	Validate and classify orders in < 10s	validate_payment(), check_vendor_sla(), classify_order_type()	validated_order_json	Return HTTP 503 + retry queue
InventoryAgent	Reserve stock across warehouses	check_stock_level(), reserve_inventory(), check_cold_chain()	inventory_reservation	Hold order; alert warehouse manager
WarehouseRouterAgent	Select optimal warehouse	calc_route_distance(), check_capacity(), estimate_sla_risk()	warehouse_assignment	Fallback to nearest warehouse by distance
DeliveryAgent	Assign carrier + generate tracking ID	select_carrier(), calculate_eta(), create_shipment()	tracking_id + ETA	Switch to backup carrier
CustomerNotifyAgent	Send order + tracking notifications	send_email(), send_sms(), push_notification()	notification_sent_status	Log failure; retry in 15 min

Key insight: For e-commerce, "fails gracefully" often means "hold the order and alert a human" — never silently skip a step or customers will lose their order.

Q5 How will agents communicate?**Communication Language:**

Structured JSON schema with typed order events. Kafka event stream for high-volume async processing (not REST — too slow at 35 orders/min).

Ontology (shared vocabulary):

Component	Domain Objects / Terms
Objects	Order, Product, Warehouse, Shipment, Carrier, Route
Actions	reserve(), route(), ship(), notify()
Attributes	order_id, sku, qty, warehouse_id, carrier_code, eta_hours, sla_deadline
Constraints	cold_chain_required → carrier_has_cold_chain = True; delivery_eta ≤ sla_deadline

FIPA-ACL Protocol — key message exchanges:

- OrderIntakeAgent → InventoryAgent: INFORM(order_id=X, items=[{sku:A, qty:2}, ...])
- InventoryAgent → WarehouseRouterAgent: INFORM(reservation_id=Y, warehouses_with_stock=[W2, W7])
- WarehouseRouterAgent → DeliveryAgent: REQUEST(ship_from=W2, order=X, sla_deadline=48h)
- DeliveryAgent → CustomerNotifyAgent: INFORM(tracking_id=T1234, eta=2026-05-14)
- Any agent → OrderIntakeAgent: FAILURE(order_id=X, reason=out_of_stock) — exception flow

Key insight: INFORM flows downstream (data passing). FAILURE flows back upstream (exception signaling). Keep these two directions distinct in your protocol design.

Q6 How do agents collaborate?**Collaboration mode and rationale:**

Fully cooperative — all agents share the goal of fulfilling each order on time. No competition within the MAS. The only competitive element is external: carriers compete for delivery contracts (handled by DeliveryAgent selecting the best carrier).

Key insight: Most fulfillment pipelines are purely cooperative. Competition is only relevant when agents must bid for shared resources — which doesn't apply here since each order has its own dedicated pipeline.

Q7 What is the agents' environment?**Observability analysis:**

Partially observable — InventoryAgent sees stock across warehouses (broad view) but DeliveryAgent sees real-time carrier availability (narrow view). Highly dynamic — flash sales change demand 10x within minutes; carrier availability changes in real time. State estimation: demand forecasting to pre-position inventory before flash sales; InventoryAgent uses safety stock model to predict stockouts 2 hours in advance.

Key insight: The 10x flash sale spike is the hardest observability challenge — inventory looks fine 5 minutes before a sale starts, then 200 orders arrive simultaneously. Predictive pre-positioning (state estimation) is the solution.

Q8 How does MAS fit your frontend and backend?**Frontend / Backend / Deployment design:**

API: POST /api/orders triggers the pipeline. GET /api/orders/{id}/status for tracking.

Real-time: Order status streamed via WebSocket to customer app (submitted → confirmed → shipped → delivered).

Deployment: GCP Cloud Run (serverless) — auto-scales from 35/min to 350/min during flash sales with zero re-architecture.

ADK Pattern: SequentialAgent([IntakeAgent, InventoryAgent, RouterAgent, DeliveryAgent, NotifyAgent]).

Monitoring: SLA breach alert if WarehouseRouterAgent stage exceeds 30 seconds. Order-level latency tracked per stage.

Key insight: For high-throughput pipelines, serverless (Cloud Run, Lambda) is the right deployment — not always Kubernetes. Match your architecture to your scaling pattern.

Q1 What specific problem are you solving?**Sample problem statement:**

An 80-bed emergency department receiving 400+ patients/day achieves only 55% compliance with triage-to-treatment time targets because nurses are overwhelmed, bed status is tracked manually on whiteboards, and concurrent record updates by multiple care teams cause medication errors. The goal is a MAS that reduces average triage-to-treatment time by 40% (from 35 to 21 minutes) and cuts medication errors by 80%, while keeping all life-critical decisions physician-confirmed.

Key insight: Healthcare problem statements must specify the HUMAN SAFETY constraint explicitly — "life-critical decisions need physician confirmation" is as important as the KPI targets.

Q2 Why do you need MAS and not a single agent?**Why MAS is justified — at least 2 of these 4 criteria apply:**

- Concurrent workstreams: vital monitoring, bed management, pharmacy, specialist scheduling all run simultaneously — sequential would mean patients wait for each step.
- Specialization: clinical decision support, bed logistics, and pharmacy each require fundamentally different data and expertise.
- Life-critical fault tolerance: if BedManagementAgent fails, TriageAgent must still function — patient triage cannot stop.
- Privacy compliance (PDPA/HIPAA): separating agents by function limits data access scope, reducing unauthorized access risk.

Key insight: Fault tolerance is a safety requirement in healthcare, not just a reliability feature. Design with the assumption that every agent WILL fail eventually.

Q3 How many agents do you need?**5 agents — one per distinct responsibility boundary:**

Agent Name	Primary Role
TriageAgent	Assesses patient severity using ESI/CTAS clinical scoring within 5 min
DiagnosticSupportAgent	Suggests differential diagnoses and orders appropriate tests
BedManagementAgent	Assigns and tracks bed availability across ED zones in real time
PharmacyAgent	Validates medication orders and checks for drug interactions before dispensing
SpecialistCoordAgent	Coordinates specialist availability and manages consult requests

Key insight: These 5 agents map to the 5 distinct workflows in a real ED. If you merged TriageAgent + DiagnosticSupportAgent, one failure would take down both — keep them separate for fault isolation.

Q4 What does each agent do?**For each agent — goal, tools, output, and failure behavior:**

Agent	Goal	Key Tools	Output	If it Fails
TriageAgent	Score patient severity in < 5 min	read_vitals(), score_esi(), check_chief_complaint()	triage_level(1-5)	Nurse manually assigns triage level (safe fallback)
DiagnosticSupportAgent	Suggest diagnostics; flag critical findings	query_ehr(), run_cds_rules(), order_lab_tests()	diagnostic_suggestion	Physician proceeds without AI support (fail-safe)
BedManagementAgent	Track and allocate beds in real time	read_bed_status(), assign_bed(), estimate_discharge_time()	bed_assignment	Nurse coordinator uses whiteboard (manual fallback)
PharmacyAgent	Validate all medication orders before dispensing	check_drug_interactions(), verify_dosage(), check_allergies()	validation_result	Pharmacist manual check required (no auto-dispense)
SpecialistCoordinator	Identify available specialist + send consult request	check_specialist_schedule(), send_consult(), track_response()	specialist_confirmed	Direct page to on-call physician

Key insight: In healthcare, every agent's failure mode must default to "safe" (human takes over), never to "skip this step." The system must be designed so human fallback is always possible.

Q5 How will agents communicate?**Communication Language:**

HL7 FHIR for clinical data exchange (international healthcare standard). FIPA ACL for agent coordination. Two separate protocols for two separate concerns.

Ontology (shared vocabulary):

Component	Domain Objects / Terms
Objects	Patient, Bed, Medication, Specialist, LabTest, TriageLevel
Actions	assess(), assign_bed(), order(), validate_medication(), consult()
Attributes	patient_id, severity_level(1-5), bed_status, medication_name, drug_interactions, specialist_eta
Constraints	Level-1 patients get bed within 5 min; all medication orders need PharmacyAgent AGREE before dispensing

FIPA-ACL Protocol — key message exchanges:

- TriageAgent → BedManagementAgent: REQUEST(patient_id=P1234, triage_level=2, bed_type=trauma)
- BedManagementAgent → TriageAgent: AGREE(bed_id=ED-23) or REFUSE(reason=no_trauma_bed)
- DiagnosticSupportAgent → PharmacyAgent: REQUEST(validate, patient=P1234, drug=morphine_5mg)
- PharmacyAgent → DiagnosticSupportAgent: INFORM(approved) or FAILURE(reason=allergy_conflict)
- SpecialistCoordAgent → All Specialists: CFP(consult_type=cardiology, urgency=immediate)
- Cardiologist: AGREE(eta=5min) or REFUSE(reason=in_surgery)

Key insight: Using both HL7 FHIR (clinical standard) and FIPA ACL (agent coordination) in the same system is correct — they serve different layers. Don't try to use one for both.

Q6 How do agents collaborate?**Collaboration mode and rationale:**

Cooperative — all agents share the goal of optimal patient care. SpecialistCoordAgent uses Contract Net Protocol (CFP) when multiple Level-1 patients simultaneously need the same specialist type — specialists "bid" on which patient they will take based on their current workload. This is a market mechanism for resource allocation within a cooperative system.

Key insight: Even in life-critical cooperative systems, resource allocation (who gets the scarce specialist) often requires a fair competitive mechanism. Contract Net is the standard pattern for this.

Q7 What is the agents' environment?**Observability analysis:**

Partially observable — TriageAgent sees one patient at a time. BedManagementAgent has the broadest view (all beds) but cannot see inside operating theatres. Highly dynamic — patients arrive unpredictably; beds change status every few minutes; specialists become available/unavailable throughout the shift. State estimation: BedManagementAgent predicts discharge times using historical length-of-stay data by diagnosis code — this is bed forecasting. Shared memory: the Electronic Health Record (EHR) is the central state store all agents read from and write to (with appropriate access controls).

Key insight: In healthcare, the EHR is the "blackboard" — a shared memory architecture where agents post their findings and read others' findings. This is a classic MAS Blackboard pattern.

Q8 How does MAS fit your frontend and backend?**Frontend / Backend / Deployment design:**

API: HL7 FHIR REST API as the interoperability layer. ED dashboard shows live patient queue, bed map, and alert feed via WebSocket.

Human trigger: Nurse registers patient at desk → triggers TriageAgent workflow. All Level-1 decisions confirmed by attending physician (Human-in-the-Loop).

Privacy: On-premise hospital server — no patient data leaves the network. Agents run in an air-gapped containerized environment.

ADK Pattern: LoopAgent(vital_sign_monitoring) running in parallel with SequentialAgent([Triage → Diagnostic → BedAssignment → Pharmacy]) per patient event. SpecialistCoordAgent as LlmAgent + transfer_to_agent().

A2A: Lab (LIMS), radiology (PACS), and pharmacy dispensing system connected via A2A with FHIR schemas.

Key insight: On-premise deployment is sometimes the ONLY option for privacy-regulated industries (healthcare, finance, government). Factor this into Q8 early — it affects your entire architecture.

Q1 What specific problem are you solving?**Sample problem statement:**

A PCB factory producing 10,000 units/day suffers a 2.3% defect rate causing \$180,000/month in waste and rework because root causes — machine calibration drift, bad material batches, operator errors — cannot be quickly identified from disconnected quality logs. The goal is a MAS that detects defects in real time, identifies the root cause within 30 minutes of anomaly onset, and reduces defect rate to below 0.5% within 3 months, while keeping total maintenance downtime within the 4-hour/month limit.

Key insight: The 30-minute root cause window and the 4-hour/month downtime limit are hard constraints — your MAS design must be evaluated against both. Always translate business constraints into agent design rules.

Q2 Why do you need MAS and not a single agent?**Why MAS is justified — at least 2 of these 4 criteria apply:**

- Three parallel production lines: VisionInspectionAgent must run simultaneously on all three — a single agent cannot maintain real-time coverage.
- Specialization: computer vision inspection, statistical root cause analysis, and production parameter adjustment are fundamentally different domains.
- Human-in-the-loop requirement: maintenance line halts need supervisor approval — this requires a dedicated decision gate in the agent flow.
- ISO 9001 traceability: audit trail generation is a separate non-functional concern — isolate it to avoid contaminating production logic.

Key insight: Compliance requirements (ISO, HIPAA, FDA) often justify adding a dedicated agent whose only job is generating regulatory artifacts — this keeps compliance logic separate from production logic.

Q3 How many agents do you need?**5 agents — one per distinct responsibility boundary:**

Agent Name	Primary Role
VisionInspectionAgent	Analyzes PCB images from AOI cameras, detects visual defects in real time
DefectClassifierAgent	Classifies defect type (solder bridge, open circuit, misalignment) and severity
RootCauseAgent	Correlates defect patterns with machine parameters, batches, and operator shifts
MaintenanceSchedulerAgent	Plans maintenance windows; escalates line-halt decisions to supervisor
ProductionAdjAgent	Adjusts production parameters (oven temp, paste volume) to reduce defect rate

Key insight: Notice that VisionInspectionAgent and DefectClassifierAgent are separate — inspection (is there a defect?) and classification (what kind of defect?) are distinct tasks with different models and latency requirements.

Q4 What does each agent do?**For each agent — goal, tools, output, and failure behavior:**

Agent	Goal	Key Tools	Output	If it Fails
VisionInspectionAgent	Detect visual defects on every board	aoi_image_reader(), defect_detection_model()	defect_event(board_id, type, image)	Manual visual inspection; line continues at 50% speed
DefectClassifierAgent	Classify defect type and severity	classifier_model(), defect_library_lookup()	defect_classification	Forward unclassified defect events to RootCauseAgent
RootCauseAgent	Identify root cause within 30 min	read_machine_logs(), read_batch_data(), read_shift_schedule(), correlation_analysis()	root_cause_hypothesis	Alert quality engineer for manual investigation
MaintenanceSchedulerAgent	Schedule maintenance within 4hr/month budget	read_maintenance_history(), find_available_window(), create_work_order()	maintenance_schedule	HITL: line halt requires supervisor approval
ProductionAdjAgent	Tune parameters to minimize defect rate	read_spc_data(), adjust_oven_profile(), adjust_paste_volume()	parameter_adjustment_log	Revert to last known good parameters

Key insight: ProductionAdjAgent's failure mode is "revert to last known good" — this is a safe default that keeps production running. In manufacturing, stopping production is itself a failure mode.

Q5 How will agents communicate?**Communication Language:**

OPC-UA protocol for machine telemetry (industrial standard — use the right protocol for your industry). JSON over REST/MQTT for inter-agent coordination.

Ontology (shared vocabulary):

Component	Domain Objects / Terms
Objects	PCBBoard, Machine, DefectEvent, MaterialBatch, MaintenanceWindow
Actions	inspect(), classify(), analyze(), schedule_maintenance(), adjust_params()
Attributes	board_id, machine_id, defect_type, defect_rate, batch_number, oven_temp_c, supervisor_approved
Constraints	maintenance_agent cannot halt line without supervisor_approved=True; total_downtime ≤ 4hr/month

FIPA-ACL Protocol — key message exchanges:

- VisionInspectionAgent → DefectClassifierAgent: INFORM(board_id=B12345, anomaly_score=0.87)
- DefectClassifierAgent → RootCauseAgent: INFORM(defect_type=solder_bridge, severity=HIGH, count_30min=47)
- RootCauseAgent → MaintenanceSchedulerAgent: REQUEST(calibrate_reflow_oven, machine_id=M7, urgency=high)
- MaintenanceSchedulerAgent → Supervisor (HITL): REQUEST(approve_halt, line=Line-2, duration=2h)
- Supervisor approves → MaintenanceSchedulerAgent → ProductionAdjAgent: INFORM(halt_approved, start=14:00)
- RootCauseAgent → ProductionAdjAgent: REQUEST(adjust, oven_profile=+3°C, paste_volume=-0.5mg)

Key insight: The Human-in-the-Loop (supervisor approval) is modeled as a special message step in the protocol — it is not separate from the communication design. Always show HITL steps explicitly in your protocol diagram.

Q6 How do agents collaborate?**Collaboration mode and rationale:**

Cooperative among all MAS agents — shared goal is quality improvement. When multiple lines simultaneously need maintenance and only one window is available, MaintenanceSchedulerAgent uses a priority queue based on defect severity — implicit competitive allocation without an explicit auction protocol.

Key insight: A priority queue is a simpler alternative to Contract Net for resource allocation when the prioritization rule is clear and deterministic. Choose the simplest mechanism that solves the problem.

Q7 What is the agents' environment?**Observability analysis:**

Partially observable — VisionInspectionAgent sees the board surface only (not electrical properties). RootCauseAgent has a broader view (machine logs + batch data + visual defects) but cannot directly observe operator fatigue. Dynamic — machine calibration drifts over time; batch quality varies per delivery. State estimation: RootCauseAgent performs statistical correlation between defect rate spikes and machine parameters, batch IDs, and shift changes — inferring root cause from indirect signals. This is implicit Bayesian reasoning.

Key insight: Root cause analysis IS state estimation — the true cause (machine drift, bad batch) is a hidden state. The agent infers it from observable symptoms (defect patterns, parameter logs).

Q8 How does MAS fit your frontend and backend?**Frontend / Backend / Deployment design:**

API: OPC-UA gateway (machine → MAS). REST API for MES/ERP integration (POST /api/defect-event, GET /api/quality-summary).

Dashboard: Real-time defect rate per line, root cause visualization, maintenance calendar with uptime budget tracker.

Deployment: Edge server in factory (sub-second latency to AOI cameras critical). MES and ERP connected via A2A.

ADK Pattern: ParallelAgent([Line1Inspector, Line2Inspector, Line3Inspector]) each running

SequentialAgent([VisionInspect → Classify → RootCause]) → MaintenanceSchedulerAgent with Human-in-the-Loop for halt approvals.

Monitoring: Agent processing time SLA: < 5 seconds per board. Alert if defect_rate > 1% for 10 consecutive boards on any line.

Key insight: Three parallel line agents (one per production line) is the right granularity — not one agent per board, not one agent for all three lines. Match agent granularity to your fault isolation requirements.

Q1 What specific problem are you solving?**Sample problem statement:**

A digital bank processing 5M transactions/day loses \$2.4M/month to fraud while its rule-based detection system simultaneously blocks 40% of legitimate transactions as false positives, damaging customer trust. The goal is a MAS that detects and responds to fraud within 200ms latency, reduces the false positive rate below 5%, and maintains 99.9% pass-through for legitimate transactions — all within regulatory reporting requirements.

Key insight: The 200ms constraint is the defining technical challenge — it dictates your architecture choices at every level (language, deployment, communication protocol). Always surface the hardest constraint in Q1.

Q2 Why do you need MAS and not a single agent?**Why MAS is justified — at least 2 of these 4 criteria apply:**

- 200ms budget forces parallelism: anomaly detection, risk scoring, and ML model inference must run concurrently — sequential execution is too slow.
- Specialization: statistical anomaly models, graph-based fraud detection, and device fingerprinting are distinct ML systems that should be developed and updated independently.
- Adversarial environment: fraud patterns evolve weekly — different fraud-type specialists (account takeover vs. synthetic identity) can be updated independently without touching the whole system.
- Regulatory compliance: audit trail generation must be isolated from the detection pipeline to avoid adding latency to the 200ms window.

Key insight: The 200ms latency constraint is the strongest technical justification for MAS here — it forces you into a parallel architecture that a single sequential agent cannot satisfy.

Q3 How many agents do you need?**5 agents — one per distinct responsibility boundary:**

Agent Name	Primary Role
TransactionMonitorAgent	Listens to real-time transaction stream; filters suspicious events for investigation
AnomalyDetectorAgent	Applies ML models to detect transaction anomalies (velocity, geo, behavioral)
RiskScorerAgent	Calculates composite fraud risk score from multiple signals with explanation
AccountActionAgent	Executes account protection (block, flag, 2FA) based on risk score
CustomerNotifyAgent	Alerts customer in real time and handles challenge authentication

Key insight: AnomalyDetectorAgent and RiskScorerAgent are separate because they run different types of computation — anomaly detection uses ML models; risk scoring aggregates signals with business rules. Different update cycles, different teams.

Q4 What does each agent do?**For each agent — goal, tools, output, and failure behavior:**

Agent	Goal	Key Tools	Output	If it Fails
TransactionMonitorAgent	Detect suspicious transactions deviating from baseline	kafka_consumer(), apply_velocity_rules(), compute_baseline_deviation()	suspicious_transaction_event	Fallback to rule-only detection
AnomalyDetectorAgent	Score transaction against learned fraud patterns	run_isolation_forest(), run_gnn(), lookup_fraud_blacklist()	anomaly_signals{velocity, geo, network}	RiskScorerAgent uses rule-based signals only
RiskScorerAgent	Produce composite risk score (0–100) with explanation	weighted_feature_scoring(), fetch_historical_behavior(), get_device_fingerprint()	risk_score + explanation	Apply conservative hold if score unavailable (fail-secure)
AccountActionAgent	Take proportionate protective action in < 200ms	block_transaction(), flag_for_review(), trigger_2fa(), freeze_account()	action_taken + audit_log	Default to block transaction (fail-secure)
CustomerNotifyAgent	Notify customer + resolve challenge	send_push_notification(), send_sms(), validate_2fa_response()	customer_response	Escalate to fraud ops team
Key insight: Both AccountActionAgent and RiskScorerAgent fail-secure (block/hold). In high-stakes financial systems, the default failure mode must always protect the customer, even if it creates friction.				

Q5 How will agents communicate?

Communication Language:

Event-driven with Kafka topics (not REST — REST cannot sustain 5M tx/day at 200ms). Structured JSON schema with typed fraud events. NO natural language — latency budget does not allow LLM inference in the hot path.

Ontology (shared vocabulary):

Component	Domain Objects / Terms
Objects	Transaction, Account, Device, FraudPattern, RiskScore, AuditLog
Actions	monitor(), score(), block(), notify(), log()
Attributes	tx_id, amount, merchant, geo_location, risk_score, fraud_type, action_taken, decision_timestamp
Constraints	total_latency(all agents) ≤ 200ms; no PII in Kafka topics — use tokenized IDs only

FIPA-ACL Protocol — key message exchanges:

- TransactionMonitorAgent → AnomalyDetector + RiskScorer: INFORM(suspicious_tx_event) [parallel Kafka fan-out]
- AnomalyDetectorAgent → RiskScorerAgent: INFORM(signals={velocity:0.9, geo:0.3, behavioral:0.7})
- RiskScorerAgent → AccountActionAgent: REQUEST(action, tx_id=X, risk_score=87, reason="high_velocity+new_device")
- AccountActionAgent → CustomerNotifyAgent: REQUEST(challenge_auth, customer_id=C456, channel=push)
- CustomerNotifyAgent → AccountActionAgent: INFORM(2fa_result=passed) or INFORM(2fa_result=timeout)

Key insight: At 200ms, you cannot use natural language between agents. Everything must be a typed JSON schema with pre-defined field names. The latency constraint directly determines the communication language.

Q6 How do agents collaborate?

Collaboration mode and rationale:

Cooperative in the detection pipeline. When multiple fraud specialist sub-models simultaneously flag a transaction and resources are limited, a weighted priority auction determines which signal gets deeper investigation first — but this happens within AnomalyDetectorAgent, not between separate MAS agents.

Key insight: Competitive mechanisms can exist within a single agent (as an internal algorithm) — they don't always need to be modeled as separate agents competing via messages. Choose the right level of decomposition.

Q7 What is the agents' environment?

Observability analysis:

Partially observable — TransactionMonitorAgent sees only transaction metadata (not account history). AnomalyDetectorAgent sees behavioral patterns but not device-level signals (separate microservice). Highly dynamic and adversarial — fraud patterns evolve weekly; fraudsters adapt to detection patterns. Uncertainty handling: RiskScorerAgent combines multiple weak signals into a probabilistic composite score — no single signal is conclusive. This is explicit reasoning under uncertainty. No agent has full observability — that is why combining multiple specialists improves accuracy over any single rule-based system.

Key insight: Combining multiple partially-observable agents often produces better decisions than one fully-observable agent — this is ensemble reasoning at the system architecture level.

Q8 How does MAS fit your frontend and backend?

Frontend / Backend / Deployment design:

API surface: Kafka topic "transactions.suspicious" (event-driven). Results written to Kafka "fraud.decisions" consumed by core banking — NOT REST, too slow.

Integration: Core banking → MAS via Kafka. MAS → Customer app via push notification gateway. MAS → Regulatory system via A2A (24-hour reporting window).

Deployment: Kubernetes with autoscaling. Each agent scales independently based on Kafka consumer lag — no single bottleneck.

ADK Pattern: ParallelAgent([AnomalyDetectorAgent, RiskScorerAgent]) → AccountActionAgent → CustomerNotifyAgent.

Monitoring: p99 latency per agent stage. False positive rate tracked daily. Alert if any stage exceeds 50ms (budget exceeded → SLA breach risk).

Key insight: REST API is wrong for 200ms latency at 5M tx/day — Kafka event streaming is the only viable architecture. Q8 forces you to make this architectural decision explicitly.

Q1 What specific problem are you solving?

Sample problem statement:

A city with 500+ intersections and siloed transport systems (signals, metro, parking) sees emergency vehicle response times averaging 9 minutes against a 6-minute target, costing \$1.2M/day in rush-hour gridlock from uncoordinated traffic management. The goal is a MAS that coordinates all transport subsystems to reduce emergency response time to ≤ 6 minutes and reduce the average congestion index by 25% within one year, without overriding manually set school zone timings.

Key insight: Two KPIs with different priorities (emergency response = hardest constraint; congestion reduction = strategic goal). This priority ordering will directly determine your Q6 (collaboration) and Q8 (ADK pattern) answers.

Q2 Why do you need MAS and not a single agent?

Why MAS is justified — at least 2 of these 4 criteria apply:

- Massive geographic distribution: 500+ intersections cannot be managed by one agent — geographic decomposition (district-level agents) is mandatory.
- Specialization: traffic signal optimization, emergency corridor routing, transit scheduling, and parking guidance each need specialized algorithms and data.
- Priority preemption: emergency routing must override all other agents in milliseconds — only a MAS with explicit priority messaging can implement this cleanly.
- Cross-organizational data: traffic (roads dept.), metro (transit authority), parking (parking corp.) are separate organizations — A2A enables cross-boundary coordination.

Key insight: Cross-organizational data sharing is a political and technical barrier that MAS + A2A specifically addresses — agents from different organizations can communicate without exposing their internal systems.

Q3 How many agents do you need?

5 agents — one per distinct responsibility boundary:

Agent Name	Primary Role
TrafficSignalAgent	Optimizes signal timing at intersection clusters based on real-time traffic flow
EmergencyRoutingAgent	Clears green corridors for emergency vehicles by coordinating preemptive signal changes
PublicTransportAgent	Adjusts bus/metro schedules and reroutes around road incidents
ParkingGuidanceAgent	Directs drivers to available parking via VMS signs and navigation apps
IncidentResponseAgent	Detects accidents/roadworks and broadcasts rerouting directives city-wide

Key insight: TrafficSignalAgent would realistically be instanced per district (e.g., 10 district agents for 500 intersections), not one agent per intersection. Always consider whether an agent should be a singleton or multiple instances.

Q4 What does each agent do?

For each agent — goal, tools, output, and failure behavior:

Agent	Goal	Key Tools	Output	If it Fails
TrafficSignalAgent	Minimize vehicle wait time at intersections in assigned district	read_loop_detector(), read_camera_flow(), set_signal_phase(), run_adaptive_algorithm()	signal_timing_plan	Intersections revert to fixed timing plan (safe default)
EmergencyRoutingAgent	Clear green corridor within 60 seconds of dispatch	get_vehicle_gps(), calc_optimal_route(), preempt_signals_on_route(), notify_district_agents()	corridor_cleared_confirmation	Manual radio coordination by traffic control center
PublicTransportAgent	Maintain transit reliability; reroute buses around incidents	read_gtfs_realtime(), read_incident_alerts(), adjust_bus_route(), notify_passengers()	updated_schedules	Transit app shows last known schedule
ParkingGuidanceAgent	Reduce circling traffic by guiding drivers to available spaces	read_parking_sensors(), find_nearest_available(), update_vms_signs(), push_navigation_update()	guidance_messages	VMS signs show last known availability
IncidentResponseAgent	Detect incidents within 3 minutes; broadcast rerouting city-wide	analyze_camera_feed(), receive_police_report(), calculate_impact_zone(), broadcast_alert()	incident_alert	Manual reporting to traffic control center

Key insight: EmergencyRoutingAgent's failure mode escalates to humans immediately — for safety-of-life systems, human fallback must be defined explicitly and must always be viable.

Q5 How will agents communicate?**Communication Language:**

MQTT for sensor streams (IoT standard). REST API for cross-agency data. JSON for agent coordination messages. Three protocols serving three different layers — this is correct.

Ontology (shared vocabulary):

Component	Domain Objects / Terms
Objects	Intersection, Vehicle, EmergencyVehicle, BusRoute, ParkingZone, Incident
Actions	set_signal(), preempt_corridor(), reroute_bus(), guide_to_parking(), broadcast_incident()
Attributes	intersection_id, phase_duration, vehicle_count, incident_type, available_spots, corridor_active
Constraints	TrafficSignalAgent cannot override manual school_zone_timing; EmergencyRoutingAgent has override priority over ALL other agents

FIPA-ACL Protocol — key message exchanges:

- IncidentResponseAgent → All Agents: INFORM(incident_id=I789, GPS, severity=HIGH) [broadcast]
- EmergencyRoutingAgent → TrafficSignalAgents (on route): REQUEST(preempt_corridor, route=[Sig12, Sig15], duration=3min)
- TrafficSignalAgent → EmergencyRoutingAgent: AGREE(corridor_clear) or REFUSE(reason=manual_override_active)
- EmergencyRoutingAgent → PublicTransportAgent: INFORM(corridor_active, affected_route=Bus64, duration=3min)
- PublicTransportAgent → ParkingGuidanceAgent: INFORM(rerouted_bus=64, expected_congestion_area=Sukhumvit)

Key insight: The IncidentResponseAgent broadcast INFORM triggers a cascade of coordinated responses across all agents — one message, five agent reactions. Broadcast INFORM is powerful for system-wide state changes.

Q6 How do agents collaborate?**Collaboration mode and rationale:**

Cooperative by default — all agents share city-wide traffic optimization goals. Hierarchical authority override: EmergencyRoutingAgent has unconditional override authority over all other agents during an active emergency corridor — this is not competition but a priority protocol built into the MAS constitution.

Key insight: Hierarchical authority is a third collaboration pattern beyond cooperative/competitive — some agents are granted unconditional override rights for safety-critical scenarios. Design this in the protocol (Q5), not just as a policy note.

Q7 What is the agents' environment?**Observability analysis:**

Partially observable — each TrafficSignalAgent sees only its district (10% of the city). IncidentResponseAgent has the broadest view (city-wide cameras) but cannot see inside parking structures. Highly dynamic — incidents are unpredictable; weather affects traffic flow; school times shift demand patterns. State estimation: EmergencyRoutingAgent uses real-time GPS tracking of the emergency vehicle to re-route dynamically if the planned corridor becomes blocked. TrafficSignalAgent uses adaptive signal control — a real-time optimization model that estimates future queue lengths from current flows.

Key insight: A city-wide system is NEVER fully observable by any single agent. The only way to get a city-wide view is to aggregate across all district agents — and by the time you do, it's already slightly outdated.

Q8 How does MAS fit your frontend and backend?**Frontend / Backend / Deployment design:**

API: City traffic platform exposes REST APIs for agencies. Cross-agency data via A2A (transit authority, parking corp., police).

Public interface: Navigation apps (Google Maps) subscribe to incident and parking APIs via the city platform.

Dashboard: Traffic control room receives real-time city map with agent decisions and active corridors via WebSocket.

Deployment: Cloud-hosted for city-wide coordination + edge nodes near critical intersections for < 100ms signal preemption latency during emergencies.

ADK Pattern: LoopAgent(5s sensor polling) → ParallelAgent([TrafficSignalAgent×10, PublicTransportAgent, ParkingGuidanceAgent]). EmergencyRoutingAgent triggered by IncidentResponseAgent with highest-priority preemption.

Key insight: Hybrid deployment (cloud + edge) is the correct architecture for latency-sensitive IoT systems: cloud for coordination and data aggregation, edge for low-latency local control.

Q1 What specific problem are you solving?**Sample problem statement:**

A B2B SaaS company handling 8,000 support tickets/day achieves only 45% first-contact resolution because senior engineers spend 60% of their time on Tier-1 issues AI could handle, while complex Tier-3 architectural problems queue for days. The goal is a MAS that auto-resolves $\geq 70\%$ of Tier-1 tickets within 15 minutes, reduces average first-response time from 4.2 hours to under 30 minutes, and increases customer satisfaction score from 3.8 to $\geq 4.3/5$.

Key insight: The "free senior engineers for complex work" goal is a business-level KPI, not just a technical one. Always frame MAS goals in business language first — cost savings, engineer time saved, customer satisfaction.

Q2 Why do you need MAS and not a single agent?**Why MAS is justified — at least 2 of these 4 criteria apply:**

- Distinct responsibilities: classification, KB retrieval, response drafting, escalation, and feedback learning are separate concerns with specialized capabilities and update cycles.
- Parallelism: 8,000 tickets/day $\approx 5.5/\text{minute}$ — concurrent processing across ticket categories is essential.
- Specialization: a classification model trained on ticket intent is very different from a retrieval model searching KB articles.
- Continuous learning loop: FeedbackLearningAgent must run asynchronously without blocking live ticket processing.

Key insight: The FeedbackLearningAgent is the "self-improvement" component of this MAS — it makes the system better over time without human intervention. This is a key argument for MAS over a simple chatbot.

Q3 How many agents do you need?**5 agents — one per distinct responsibility boundary:**

Agent Name	Primary Role
IntentClassifierAgent	Classifies ticket by type, product area, urgency, and tier within 10 seconds
KnowledgeBaseAgent	Retrieves top-3 relevant KB articles, past resolutions, and runbooks
ResponseDraftAgent	Generates personalized, technically accurate draft response using KB context
EscalationAgent	Routes complex tickets to the appropriate engineer tier within SLA
FeedbackLearningAgent	Learns from resolved tickets to improve classification accuracy and KB coverage

Key insight: FeedbackLearningAgent is the most often forgotten agent in support MAS designs — but without it, the system cannot improve and will gradually degrade as products evolve.

Q4 What does each agent do?**For each agent — goal, tools, output, and failure behavior:**

Agent	Goal	Key Tools	Output	If it Fails
IntentClassifierAgent	Classify every ticket in < 10s	nlp_intent_model(), read_ticket_metadata(), check_product_version()	ticket_classification(tier, product, urgency)	Default to Tier-2 routing (conservative fail-safe)
KnowledgeBaseAgent	Retrieve top-3 articles in < 5s	semantic_search_kb(), lookup_known_issues(), search_past_tickets()	knowledge_snippets + source_links	ResponseDraftAgent drafts from LLM knowledge only (no KB citations)
ResponseDraftAgent	Write accurate, empathetic response tailored to customer tech level	llm_response_generator(), format_code_blocks(), check_tenant_context()	draft_response	Send auto-acknowledgment with ETA only
EscalationAgent	Route non-resolvable tickets to right engineer within SLA	check_engineer_availability(), calc_escalation_priority(), create_linear_ticket()	escalation_assignment	Alert support manager
FeedbackLearningAgent	Improve classification + KB coverage continuously	analyze_resolution_outcomes(), flag_kb_gaps(), retrain_intent_model(), create_kb_draft()	updated_model + new_kb_articles	Manual KB update process (no live system impact)

Key insight: ResponseDraftAgent's failure mode (send acknowledgment with ETA) is elegant — the customer still gets a response, just not an AI-generated answer. Graceful degradation maintains customer experience even under failure.

Q5 How will agents communicate?**Communication Language:**

Natural language for customer-facing responses. Structured JSON for internal agent coordination. Two languages in one system — one external, one internal.

Ontology (shared vocabulary):

Component	Domain Objects / Terms
Objects	Ticket, KBArticle, Engineer, Resolution, CustomerAccount, FeedbackSignal
Actions	classify(), retrieve(), draft(), escalate(), learn()
Attributes	ticket_id, tier(1-3), product_area, urgency_score, customer_plan, resolution_status, csat_score
Constraints	Customer data cannot be shared between tenants; Tier-3 escalation requires human engineer

FIPA-ACL Protocol — key message exchanges:

- IntentClassifierAgent → KnowledgeBaseAgent + ResponseDraftAgent: INFORM(tier=1, product=billing_api) [parallel fan-out]
- KnowledgeBaseAgent → ResponseDraftAgent: INFORM(articles=[KB-245, KB-312], confidence=0.88)
- ResponseDraftAgent → EscalationAgent: REQUEST(review_draft) [for borderline Tier-1/2 cases]
- EscalationAgent → ResponseDraftAgent: AGREE(auto_resolve=true) or REQUEST(escalate, tier=2)
- After resolution: ResponseDraftAgent → FeedbackLearningAgent: INFORM(outcome=resolved, csat=4.2, time=8min)

Key insight: The feedback loop (resolution → FeedbackLearningAgent → model update) is what separates a static chatbot from a learning MAS. Design this loop explicitly in your communication protocol.

Q6 How do agents collaborate?**Collaboration mode and rationale:**

Fully cooperative — all agents share the goal of maximizing resolution quality and speed for every ticket. No competition within the MAS. EscalationAgent allocates engineer capacity using a priority queue (urgency × customer tier), but this is an internal prioritization algorithm, not a competitive agent interaction.

Key insight: Customer support MAS are almost always cooperative — there is no benefit to competition within the pipeline. Reserve competitive/market mechanisms for external resource allocation (like specialist scheduling in healthcare).

Q7 What is the agents' environment?**Observability analysis:**

Partially observable — IntentClassifierAgent sees only ticket text (not account history). KnowledgeBaseAgent sees the KB but not the customer's specific product configuration. Mostly static per ticket (a ticket doesn't change while agents process it), but the KB evolves continuously. Key shared state: customer account context (subscription tier, product version, past tickets) must be retrieved from CRM — this is the most important shared state for personalization. Memory: FeedbackLearningAgent needs long-term episodic memory of all resolved tickets for its retraining corpus.

Key insight: CRM data is the "ground truth" shared state for customer support MAS — without it, agents cannot personalize responses or detect repeat issues. Always identify your system's central state store early.

Q8 How does MAS fit your frontend and backend?**Frontend / Backend / Deployment design:**

API: Zendesk webhook triggers MAS on new ticket → POST /api/tickets/process. Response posted back to Zendesk via Zendesk API. Customer sees the Zendesk interface — MAS is invisible.

Human approval: ResponseDraftAgent submits draft to Zendesk; human agent can edit before sending, or approve auto-send for Tier-1 with confidence > 0.9.

Dashboard: Engineer queue shows escalated tickets with AI triage reason, KB articles consulted, and resolution confidence score.

Deployment: AWS Lambda for IntentClassifier (fast, cheap, auto-scales). SQS queue for async processing.

ADK Pattern: IntentClassifierAgent (LlmAgent + transfer_to_agent) dispatches to ParallelAgent([KnowledgeBaseAgent, ResponseDraftAgent]) → EscalationAgent gate.

A2A: Zendesk, Linear, Slack, and CRM connected via A2A.

Key insight: The MAS is hidden behind Zendesk — customers and engineers interact through existing tools. This is the right approach: augment existing workflows, don't replace them. Lower adoption barrier.

Q1 What specific problem are you solving?**Sample problem statement:**

A global electronics manufacturer sourcing from 200 suppliers in 15 countries has suffered \$45M in disruption losses from 3 major events (typhoon, factory fire, port strike), with an average detection lag of 72 hours and 2 weeks needed for manual re-routing. The goal is a MAS that detects supply disruptions within 2 hours of event onset, completes re-routing recommendations within 4 hours, and reduces exposure by 60% — with procurement manager approval required for contracts above \$100k.

Key insight: The \$100k approval threshold is a business policy constraint that directly shapes the agent communication protocol (HITL step). Always extract policy constraints from the business description and put them in Q1.

Q2 Why do you need MAS and not a single agent?**Why MAS is justified — at least 2 of these 4 criteria apply:**

- Scale of monitoring: 200 suppliers × 15 countries need continuous parallel monitoring of news, weather, geopolitical events, and shipping — impossible for one agent.
- Specialization: risk monitoring, supplier auditing, logistics re-routing, demand forecasting, and negotiation require different data sources and expertise.
- Concurrent investigation: during a disruption, supply impact, logistics alternatives, and demand revision must all be assessed simultaneously within the 4-hour window.
- Contract Net negotiation: NegotiationAgent must simultaneously engage multiple alternate suppliers — this requires a proper auction protocol, not sequential outreach.

Key insight: The 4-hour response window is the key MAS justification here — sequential investigation (assess one supplier, then plan logistics, then forecast demand) would take 12+ hours. Parallelism is the only way to hit 4 hours.

Q3 How many agents do you need?**5 agents — one per distinct responsibility boundary:**

Agent Name	Primary Role
RiskMonitorAgent	Continuously scans news, weather, and geopolitical feeds for supply risk signals
SupplierAuditAgent	Assesses disruption impact on specific suppliers (capacity, buffer stock, alternatives)
LogisticsRerouteAgent	Identifies and evaluates alternative logistics paths and ports
DemandForecastAgent	Adjusts short-term production demand forecast based on disruption scope
NegotiationAgent	Engages alternative suppliers via CFP, evaluates bids, proposes contracts

Key insight: RiskMonitorAgent is the "early warning" agent that triggers the others. Without it, the whole system is reactive (72-hour lag). The continuous monitor pattern is what transforms the company from reactive to proactive.

Q4 What does each agent do?**For each agent — goal, tools, output, and failure behavior:**

Agent	Goal	Key Tools	Output	If it Fails
RiskMonitorAgent	Detect supply-relevant events within 2 hours	news_api_stream(), weather_alert_api(), port_status_api(), social_monitor()	risk_alert(event_type , severity, affected_regions)	Daily manual risk review by procurement team
SupplierAuditAgent	Assess per-supplier disruption impact within 30 min	read_supplier_profile(), query_erp_inventory(), contact_supplier_portal(), estimate_recovery_time()	supplier_impact_assessment	Conservative assumption: 100% disruption for affected suppliers
LogisticsRerouteAgent	Identify alternative routes/ports within 60 min	calc_alternate_routes(), check_carrier_availability(), estimate_cost_delta(), check_customs_requirements()	ranked_reroute_options	Manual logistics broker engagement
DemandForecastAgent	Revise production forecast for supply constraints	run_demand_model(), adjust_bom_requirements(), calc_safety_stock_depletion()	revised_production_plan	Production planner uses last known forecast
NegotiationAgent	Secure alternative supply contracts within 4 hours	send_rfq(), evaluate_supplier_bids(), draft_contract(), submit_for_approval()	contract_proposals + escalations	HITL: contracts > \$100k require procurement manager approval

Key insight: SupplierAuditAgent's conservative failure mode (assume 100% disruption) is intentionally pessimistic — in supply chain, it is better to over-prepare than to under-estimate the disruption and run out of stock.

Q5 How will agents communicate?**Communication Language:**

EDI for supplier communications (industry standard). JSON over REST for internal agent coordination. FIPA ACL for negotiation protocol. Three protocols for three different interaction types.

Ontology (shared vocabulary):

Component	Domain Objects / Terms
Objects	Supplier, ShipmentRoute, ProductComponent, Contract, DisruptionEvent
Actions	monitor(), audit_supplier(), reroute(), forecast_demand(), negotiate()
Attributes	supplier_id, component_sku, buffer_days, route_cost_delta, disruption_severity, contract_value_thb
Constraints	contract_value > 100000 THB → requires procurement_manager_approval; cannot cancel contracts unilaterally

FIPA-ACL Protocol — key message exchanges:

- RiskMonitorAgent → SupplierAuditAgent + LogisticsRerouteAgent + DemandForecastAgent: INFORM(disruption_event) [parallel broadcast]
- SupplierAuditAgent → NegotiationAgent: INFORM(affected_suppliers, critical_components, buffer_days=5)
- NegotiationAgent → Alt. Suppliers: CFP(task=supply_10000_units_SKU789, delivery_by=2026-05-20)
- Supplier A: PROPOSE(price=450, lead_time=7d); Supplier B: PROPOSE(price=480, lead_time=5d)
- NegotiationAgent → Supplier B: ACCEPT-PROPOSAL; → Supplier A: REJECT-PROPOSAL
- NegotiationAgent → Procurement Manager (HITL): REQUEST(approve, contract_value=5.76M THB)

Key insight: The full Contract Net Protocol (CFP → PROPOSE → ACCEPT/REJECT) is the correct pattern for competitive supplier negotiation. This is one of the most important FIPA protocols to know for business MAS.

Q6 How do agents collaborate?**Collaboration mode and rationale:**

Cooperative internally — all company agents share the goal of supply chain continuity. Competitive externally — alternative suppliers compete against each other for the contract via Contract Net. This mixed cooperation-competition pattern is very common in procurement and logistics MAS.

Key insight: Cooperation is INTERNAL (agents within the company's MAS). Competition is EXTERNAL (suppliers competing for your business). The boundary between internal and external defines where you use cooperative vs. competitive protocols.

Q7 What is the agents' environment?**Observability analysis:**

Partially observable — RiskMonitorAgent sees public event signals (news, weather) but not supplier internal capacity. SupplierAuditAgent queries suppliers directly but may receive incomplete or delayed information (suppliers may hide problems). Highly dynamic — disruptions evolve; typhoons track across multiple regions; port strikes spread. Uncertainty modeling: DemandForecastAgent produces base/bull/bear scenarios under disruption — explicit probabilistic reasoning under uncertainty. RiskMonitorAgent uses a risk score model to estimate probability of disruption from weak signals before the event is confirmed.

Key insight: Supply chain observability is fundamentally limited — you cannot know what is happening inside a supplier's factory. Design your agents to make good decisions under this information constraint, not to wait for perfect information.

Q8 How does MAS fit your frontend and backend?**Frontend / Backend / Deployment design:**

API: GET /api/supply-risk/status (procurement dashboard). POST /api/disruption/alert (manual trigger). WebSocket for live disruption world map.

Human workflow: RiskMonitorAgent alert → Slack notification to procurement manager → dashboard review → approve re-routing > \$100k via mobile approval workflow.

Reporting: Automated disruption response report emailed to supply chain leadership within 4 hours, including: affected suppliers, selected alternatives, cost delta, risk assessment.

Deployment: GCP (global reach for international APIs). NegotiationAgent interacts with supplier portals via A2A.

ADK Pattern: LoopAgent(RiskMonitorAgent) → triggers ParallelAgent([SupplierAudit, LogisticsReroute, DemandForecast]) → NegotiationAgent (with HITL for large contracts).

Key insight: The LoopAgent → ParallelAgent → SequentialAgent chain is the standard pattern for "monitor → investigate in parallel → decide" workflows. Memorize this pattern — it appears in many real business MAS.

Q1 What specific problem are you solving?**Sample problem statement:**

A real estate investment fund with \$500M AUM misses 30% of investment opportunities because manual due diligence takes 3 weeks per property while competitors with automated pipelines close deals within 3 days. The goal is a MAS that screens each property across 4 dimensions (market, legal, financial, ESG) in parallel and produces an investment recommendation with full supporting analysis within 72 hours — with licensed lawyer sign-off required on all legal findings.

Key insight: 3 weeks → 72 hours is a 10× improvement, and the parallelism of the 4 assessment dimensions is the entire business case for MAS here. Make this explicit in Q1 so it directly motivates Q2.

Q2 Why do you need MAS and not a single agent?**Why MAS is justified — at least 2 of these 4 criteria apply:**

- Natural parallel decomposition: market analysis, legal review, financial modeling, and ESG scoring are completely independent — they can run simultaneously, cutting 3 weeks to 72 hours.
- Specialization: each dimension requires different data sources, different models, and different professional expertise (market analyst, lawyer, financial modeler, ESG specialist).
- Regulatory requirement: legal findings require licensed lawyer review (Human-in-the-Loop) — this compliance gate must be isolated in its own agent.
- Data quality management: ESG data is unreliable from third-party sources — a dedicated ESGScorerAgent can cross-check and validate sources independently.

Key insight: This is the purest example of why MAS is justified — 4 completely independent workstreams that must all complete before a decision can be made. Without MAS, you run them sequentially. With MAS, you run them in parallel.

Q3 How many agents do you need?**5 agents — one per distinct responsibility boundary:**

Agent Name	Primary Role
MarketResearchAgent	Analyzes comparable sales, rental yields, location demand, and competitive supply
PropertyValuationAgent	Estimates property value using DCF, comparable, and income capitalization methods
LegalDueDiligenceAgent	Reviews title documents, zoning compliance, encumbrances, and regulatory approvals
FinancialModelAgent	Builds IRR, NPV, cash-flow model, stress-tests under base/bull/bear scenarios
ESGScorerAgent	Evaluates environmental sustainability, social impact, and governance of the property

Key insight: Note that PropertyValuationAgent is separate from FinancialModelAgent — valuation is "what is this property worth?" while financial modeling is "what return will we get if we buy it?" Different questions, different methodologies.

Q4 What does each agent do?**For each agent — goal, tools, output, and failure behavior:**

Agent	Goal	Key Tools	Output	If it Fails
MarketResearchAgent	Produce market context report within 4 hours	land_registry_api(), comparable_sales_db(), rental_market_data(), demographic_trends_api()	market_research_report(demand_score, supply_risk, location_grade)	Flag as incomplete; analyst adds manual research
PropertyValuationAgent	Produce valuation range (3 methods) within 6 hours	dcf_model(), comparable_transaction_lookup(), income_capitalization_calc()	valuation_report(low/mid/high THB)	Use comparable-only valuation as fallback
LegalDueDiligenceAgent	Surface all title risks and regulatory issues within 12 hours	land_title_search(), zoning_lookup(), encumbrance_check()	legal_risk_summary	Halt deal pending manual legal review (safety stop)
FinancialModelAgent	Produce base/bull/bear IRR and NPV within 6 hours	cash_flow_model(), capex_estimator(), debt_service_calc(), scenario_simulator()	financial_model(irr, npv, payback_period)	Analyst uses template financial model manually
ESGScorerAgent	Produce ESG composite score with source citations within 4 hours	sustainability_ratings_api(), energy_efficiency_estimator(), social_impact_checker()	esg_report(score, breakdown, risks)	Flag as unavailable; committee reviews without ESG score

Key insight: LegalDueDiligenceAgent is the only agent whose failure mode is a hard STOP (halt the deal). This is because legal risk is binary — you either have a clean title or you don't. Other dimensions can proceed with incomplete data.

Q5 How will agents communicate?

Communication Language:

Structured JSON reports for inter-agent data exchange. FIPA PROPOSE/ACCEPT-PROPOSAL for the recommendation workflow (each agent proposes its report, ConsolidationAgent accepts).

Ontology (shared vocabulary):

Component	Domain Objects / Terms
Objects	Property, MarketReport, LegalReport, FinancialModel, ESGReport, InvestmentRecommendation
Actions	research(), valuate(), audit_legal(), model_financial(), score_esg(), consolidate(), recommend()
Attributes	property_id, address, irr_base_case, legal_risk_level(low/med/high/stop), esg_score, location_grade, recommendation(BUY/HOLD/PASS)
Constraints	legal_risk_level=stop → halt deal; legal findings require licensed_lawyer_reviewed=True

FIPA-ACL Protocol — key message exchanges:

- All 4 specialist agents → ConsolidationAgent: PROPOSE(report_type, content, confidence, data_freshness) [parallel completion]
- ConsolidationAgent → Investment Committee: REQUEST(decision, property_id=P101, recommendation=BUY)
- LegalDueDiligenceAgent → Licensed Lawyer (HITL): REQUEST(review, flagged_risks=[...])
- Lawyer → LegalDueDiligenceAgent: INFORM(review_complete, status=approved)
- Investment Committee → ConsolidationAgent: AGREE(decision=approved) or REQUEST(clarification)

Key insight: Using PROPOSE for specialist reports (not INFORM) signals that the ConsolidationAgent can accept or request revision — it is not just passively receiving data. Choose performatives that model the actual authority relationship.

Q6 How do agents collaborate?

Collaboration mode and rationale:

Cooperative — all agents collaborate to produce the most accurate investment recommendation. Independence of analysis (each agent works without seeing others' results) reduces cognitive bias — this is cooperative in goal but isolated in process, similar to how professional due diligence teams work in practice.

Key insight: Information isolation between parallel agents is sometimes INTENTIONAL — it prevents one agent's findings from biasing another's. Cooperative goal, isolated execution. This is a deliberate design choice, not a limitation.

Q7 What is the agents' environment?

Observability analysis:

Partially observable by design — MarketResearchAgent sees market trends but not the property's physical condition. LegalDueDiligenceAgent sees documents but not financial projections. Information isolation between agents reduces confirmation bias in each analysis. Relatively static per deal — a deal doesn't change while agents are analyzing (72-hour window). External data staleness: each agent must timestamp its data and flag if source data is > 7 days old. Uncertainty range: FinancialModelAgent produces base/bull/bear scenarios — explicit confidence intervals, not single-point estimates.

Key insight: Producing confidence ranges (low/mid/high valuation; base/bull/bear IRR) is a form of explicit uncertainty modeling. Always ask: is the agent's output a single value or a distribution? For financial decisions, always use a distribution.

Q8 How does MAS fit your frontend and backend?

Frontend / Backend / Deployment design:

API: POST /api/deals/{property_id}/analyze triggers the parallel analysis. GET /api/deals/{id}/reports returns all completed reports.

Deal dashboard: Analysts see live completion status of each agent's report with a progress bar and estimated completion time.

Approval gate: Investment committee receives deal package via portal; approves/rejects with comments. Lawyer review portal is a separate secure tenant.

Deployment: AWS cloud. LegalDueDiligenceAgent runs in a separate security boundary (encrypted storage, audit log).

A2A connects land registry, valuation databases, and ESG data providers.

ADK Pattern: ParallelAgent([MarketResearch, LegalDueDiligence, FinancialModel, ESGScorer]) →

SequentialAgent([ConsolidationAgent, HITL(InvestmentCommittee)]).

Key insight: Separating the legal agent into its own security boundary (encrypted storage, separate audit log) is a compliance requirement, not just a technical preference. Always ask: does any agent handle data that requires special security treatment?

Q1 What specific problem are you solving?**Sample problem statement:**

A 200-location Thai restaurant chain has suffered \$2M in liability and brand damage from 3 foodborne illness incidents linked to falsified temperature logs and reactive outbreak detection (learning of issues from customer complaints, not sensors). The goal is a MAS that provides continuous, automated food safety monitoring across all 200 locations, detects temperature violations within 15 minutes, identifies potential outbreak patterns within 2 hours, and generates FDA-compliant XML reports automatically — with GM approval required before any branch closure.

Key insight: The root cause here is "reactive monitoring" — the company detects problems AFTER harm occurs. A MAS shifts this to proactive, continuous, automated monitoring. This "proactive vs. reactive" framing should be in Q1.

Q2 Why do you need MAS and not a single agent?**Why MAS is justified — at least 2 of these 4 criteria apply:**

- Scale: 200 locations × multiple sensors each = thousands of readings per minute — parallel monitoring is mandatory.
- Different execution patterns: TempMonitorAgent is a continuous loop; SupplierAuditAgent is event-triggered; RegulatoryReportAgent runs on a monthly schedule — different timing patterns need different agent types.
- Specialization: IoT sensor monitoring, food safety auditing, outbreak pattern detection, and FDA XML formatting are very different domains.
- Regulatory isolation: RegulatoryReportAgent must be isolated so compliance reporting continues even if other agents fail.

Key insight: Agents with DIFFERENT EXECUTION TIMING (continuous loop vs. monthly batch vs. event-triggered) are one of the strongest justifications for MAS — you cannot mix these timing patterns in a single agent cleanly.

Q3 How many agents do you need?**5 agents — one per distinct responsibility boundary:**

Agent Name	Primary Role
TempMonitorAgent	Continuously reads IoT temperature sensors at all locations; detects violations in ≤15 min
SupplierAuditAgent	Schedules and tracks supplier food safety audits; flags non-compliant suppliers
InspectionSchedulerAgent	Schedules branch safety inspections based on risk scoring
OutbreakDetectorAgent	Correlates complaint patterns + temp violations to detect potential outbreaks in ≤2 hrs
RegulatoryReportAgent	Generates and submits FDA-compliant XML safety reports on schedule

Key insight: TempMonitorAgent would realistically be instantiated per cluster (e.g., 20 agents for 200 locations = 10 locations each). One agent for 200 locations × 5 sensors = 1,000 data points every 5 minutes — too much for one agent.

Q4 What does each agent do?**For each agent — goal, tools, output, and failure behavior:**

Agent	Goal	Key Tools	Output	If it Fails
TempMonitorAgent	Detect violations within 15 min (per cluster of 10 locations)	mqtt_sensor_reader(), compare_to_threshold(), calc_violation_duration(), alert_branch_manager()	violation_event or compliance_report	Alert branch manager for manual log; escalate to InspectionScheduler
SupplierAuditAgent	Ensure 100% annual audit coverage of all food suppliers	read_supplier_list(), schedule_audit(), record_audit_results(), flag_non_compliant()	audit_report + compliance_status	Procurement manager manually schedules audit
InspectionScheduler Agent	Schedule branch inspections based on risk score	calc_branch_risk_score(), assign_inspector(), track_completion(), integrate_health_dept()	inspection_schedule	Default fixed inspection schedule applied
OutbreakDetectorAgent	Detect outbreak clusters within 2 hours	analyze_complaint_patterns(), correlate_with_temp_violations(), check_supplier_overlap(), spatial_clustering()	outbreak_alert + affected_locations	HITL: GM approval before branch closure
RegulatoryReportAgent	Submit FDA-compliant XML reports on schedule	generate_fda_xml(), validate_schema(), submit_to_fda_portal(), archive_report()	submission_confirmation	Alert compliance officer for manual submission

Key insight: OutbreakDetectorAgent triggers the most serious action (branch closure) and therefore has the most critical HITL gate. The severity of the action determines the strictness of the human approval requirement.

Q5 How will agents communicate?**Communication Language:**

MQTT for IoT sensor streams. JSON for inter-agent coordination. FDA-specified XML for regulatory reporting. Three different formats for three different consumers — this is correct and expected in enterprise systems.

Ontology (shared vocabulary):

Component	Domain Objects / Terms
Objects	Location, Sensor, FoodBatch, Supplier, Inspection, OutbreakCluster, FDAReport
Actions	monitor(), audit(), schedule_inspection(), detect_outbreak(), generate_report()
Attributes	location_id, sensor_id, temp_reading_c, threshold_c, batch_number, violation_duration_min, complaint_count, cluster_confidence
Constraints	temp_storage must be $\leq 4^{\circ}\text{C}$; temp_hot_hold must be $\geq 60^{\circ}\text{C}$; branch_closure requires gm_approved=True

FIPA-ACL Protocol — key message exchanges:

- TempMonitorAgent → OutbreakDetectorAgent: INFORM(location_id=BKK-023, sensor=cold_storage, temp=12°C, duration=45min)
- TempMonitorAgent → InspectionSchedulerAgent: REQUEST(urgent_inspection, location_id=BKK-023, reason=temp_violation)
- SupplierAuditAgent → OutbreakDetectorAgent: INFORM(supplier_id=S12, non_compliant=[cold_storage_temp, pest_control])
- OutbreakDetectorAgent → RegulatoryReportAgent: INFORM(cluster_id=C001, locations=[BKK-023, BKK-047], confidence=0.85)
- OutbreakDetectorAgent → GM (HITL): REQUEST(approve_closure, location_id=BKK-023)
- RegulatoryReportAgent → FDA Portal (A2A): SUBMIT(report_xml, period=monthly)

Key insight: The SUBMIT performative to the FDA Portal (A2A) represents an EXTERNAL action that crosses organizational boundaries. This is where A2A protocol applies — the FDA portal is not under your MAS control.

Q6 How do agents collaborate?**Collaboration mode and rationale:**

Cooperative — all agents share the goal of food safety compliance and outbreak prevention. InspectionSchedulerAgent allocates limited inspector resources (e.g., 5 inspectors for 200 locations) using a weighted risk-score priority queue — the highest-risk locations get inspected first. This is implicit prioritization, not agent competition.

Key insight: A priority queue for resource allocation is simpler and more transparent than a Contract Net auction when the priority criteria are clear and non-negotiable (risk score). Use the simplest mechanism that works.

Q7 What is the agents' environment?**Observability analysis:**

Partially observable — TempMonitorAgent sees sensor data but not employee hygiene or ingredient freshness. OutbreakDetectorAgent sees complaints + temperature data but not patient medical records from hospitals (external health system). Dynamic — temperature violations happen at any time; outbreaks spread across locations over hours. State estimation: OutbreakDetectorAgent uses spatial-temporal clustering to detect outbreak patterns before they are confirmed — it infers a hidden state (active outbreak) from observable signals (complaints + violations + supplier overlap). Memory needed: TempMonitorAgent needs 30-day historical temp logs per sensor for trend analysis (is a violation a one-time spike or a worsening trend?).

Key insight: Spatial-temporal clustering (are complaints from the SAME locations using the SAME suppliers in the SAME time window?) is the key state estimation technique for outbreak detection. The hidden state is "is an outbreak active?" — inferred from multiple weak signals.

Q8 How does MAS fit your frontend and backend?

Frontend / Backend / Deployment design:

API: GET /api/safety/status (compliance dashboard). GET /api/locations/{id}/violations (branch manager view). POST /api/outbreak/alert (manual trigger).

Dashboards: Chain-wide compliance map (green/yellow/red per location). Branch managers see their own sensor readings in real time on a mobile app.

Regulatory: RegulatoryReportAgent posts FDA-specified XML via A2A connection to FDA reporting portal. Archived locally for 5-year compliance record.

Deployment: Azure cloud + MQTT broker for IoT. 20 TempMonitorAgent instances (10 locations each). Scales from 200 to 2,000 locations by adding instances.

ADK Pattern: LoopAgent(TempMonitorAgent, every 5 min) → on violation: SequentialAgent([OutbreakDetectorAgent, HITL(GM_Approval)]). SupplierAuditAgent and RegulatoryReportAgent run as independent scheduled LlmAgent instances.

Key insight: Instanting TempMonitorAgent (20 instances × 10 locations) instead of 200 separate agents or 1 overloaded agent is the right granularity. Always think about horizontal scalability in Q8.

Workshop Reflection — Example Answers

These sample answers are drawn from common participant responses across multiple workshop runs. They are not the only correct answers.

QR 1 Which scenario surprised you most? Why?

Example answer A (common response):

Scenario 10 (Food Safety) surprised me most. I initially assumed a single IoT dashboard would solve this — just show temperature readings. But when I worked through Q5 (communication protocol), I realized three completely different systems must interact: MQTT IoT sensors, JSON agent coordination, AND FDA-specified XML for reporting. A single agent trying to bridge all three formats while simultaneously detecting outbreak patterns would be unmaintainable. The agent boundary became obvious once I saw the data format boundaries.

Example answer B:

Scenario 5 (Fraud Detection) was unexpected — the 200ms constraint completely changes every design decision. You cannot use natural language between agents (too slow). You cannot use REST APIs (too slow). Even the communication protocol becomes a pure typed JSON event stream. The latency constraint is the architecture.

Key insight: Surprise often comes from a CONSTRAINT you hadn't considered (latency, data format, compliance). The constraint is the design driver — always find it in Q1.

QR 2 Which scenario would you most want to implement? Which ADK pattern?

Example answer (developer with web/SaaS background):

Scenario 7 (Customer Support) because it is immediately deployable with existing tools. Zendesk webhook → IntentClassifierAgent (ADK LlmAgent + transfer_to_agent()) dispatches to ParallelAgent([KnowledgeBaseAgent, ResponseDraftAgent]) → EscalationAgent gate. The FeedbackLearningAgent runs nightly as a standalone LoopAgent. I would start with just Q1 + Q2 in the pipeline (classification + escalation) and add KB retrieval and response drafting in week 2.

Example answer (developer at an IoT/smart building company):

Scenario 1 (Hotel Energy) applies directly to our product. I would use LoopAgent(60s cycle) for continuous sensor monitoring, with ParallelAgent([OccupancyAgent, WeatherAgent]) feeding EnergyBudgetAgent, which then controls ParallelAgent([HVACControlAgent, LightingAgent]). The Contract Net protocol for load-shedding is the most interesting part to implement.

Key insight: The best answer references a SPECIFIC ADK pattern (not just "I would use MAS"). If you cannot name the ADK pattern, go back and re-read Q8 of your chosen scenario.

QR 3 What was the hardest design question across all scenarios?

Most common answer across workshop runs:

Q5 (Communication Protocol) is consistently the hardest question. Most participants can name agents (Q3) and describe what they do (Q4), but designing the shared ontology — the vocabulary that agents must agree on before any communication is possible — requires thinking at the system level, not the individual agent level. Getting the ontology wrong means agents "talk" but don't understand each other.

Second most common answer:

Q3 (How many agents?) is hard because the temptation is to add too many. "One agent per sensor," "one agent per country," "one agent per rule" — these lead to systems with hundreds of agents and unmanageable coordination overhead. The discipline of asking "Could two agents be merged without losing specialization?" is the key skill.

Key insight: Q5 is hard because ontology design is a systems-thinking skill, not a coding skill. You are defining a shared language before writing a single line of code.

QR 4 What is the most important lesson you take away?

Example answer (most impactful lesson):

The 8 design questions are a forcing function that makes you make decisions you would normally defer until implementation — and by then it is expensive to change. Q2 (Why MAS?) forced me to justify every agent. When I challenged my first design, I realized 3 of my 7 agents could be simple Python functions. The real MAS only needed 4 agents. More agents is not better — each additional agent adds coordination overhead.

Example answer (second perspective):

The biggest lesson from Q5 (communication) and Q7 (environment): agents in real systems are always partially observable. You never have perfect information. The MAS works not because one agent has complete knowledge, but because multiple agents with complementary partial views together make better decisions than any single agent with full context. Designing for partial observability from the start — not as an afterthought — is the mark of a mature MAS designer.

Key insight: The best lesson statements are personal and specific — they reference a moment in the workshop where a design decision changed. Generic statements ("MAS is powerful") are less useful than specific insights ("Q2 taught me to challenge every agent's existence").

Common Design Mistakes — and How to Fix Them

Mistake	Symptom	Fix
Too many agents	Added one agent per sensor/per country/per rule.	Merge agents that always run together with the same inputs and outputs. Ask: would merging these two agents break fault isolation or specialization?
Vague problem statement (Q1)	"Improve efficiency" with no KPI.	Always include: current metric value, target metric value, and time frame. "Reduce defect rate from 2.3% to 0.5% within 3 months."
Skip justifying MAS (Q2)	Jumped straight to agent design without proving MAS is needed.	Answer the 4 criteria: Is it too large? Needs parallelism? Needs specialization? Needs fault tolerance?
No ontology in Q5	Described messages without defining the shared vocabulary.	Always define Objects, Actions, and Attributes before designing message flows.
INFORM everywhere in Q5	Used INFORM for every message type.	INFORM = share a fact. REQUEST = ask for action. CFP = broadcast for bids. PROPOSE = respond to CFP. Choose correctly.
Ignoring failure modes (Q4)	Left "If it fails" blank.	Every agent must have a defined failure mode. Default to: "system degrades gracefully, human is notified."
Wrong ADK pattern (Q8)	Used SequentialAgent when steps are independent.	Sequential = steps depend on previous output. Parallel = steps are independent. LoopAgent = continuous repetition. Match pattern to your control flow.

Next step: Compare your own worksheet designs to these example answers. Every DIFFERENCE is a learning opportunity — ask yourself: "Is my approach also valid, or did I miss something?" Open the MAS Coding Exercises PDF and implement your favourite scenario.